

UNIVERSIDAD DE GUADALAJARA

Centro Universitario de los Altos

6^{to} semestre

Fase 1: Proyecto Final



PRESENTAN:

Kevin Josué Barba Ramirez 219486087

Luis Javier Garnica Escamilla 222967436

Ximena Ledezma Ríos 222967363

Alejandra Estefanía Martínez Muñoz 222967304

Ángel Eduardo Muñoz Pérez 217588427

Prof. Carlos Javier Cruz Franco

Materia:

Interacción Humano-Computadora

Fecha: 06/04/2025

Tabla de contenidos

Modelo matemático pluma de estacionamiento v.2	3
1. Problemática:.....	3
2. Selección del modelo:.....	3
3. Identificación de componentes.	4
4. Cuantificación.....	4
5. Calibración.....	5
6. Validación.	6
7. Análisis de sensibilidad.....	7

“Modelo matemático pluma de estacionamiento v.2”

Nuestro proyecto final consiste en hacer una mejora a la actividad anterior en la que realizamos una pluma de estacionamiento que detectaba cuándo un vehículo se aproximara y de esta manera la pluma se elevaba, sin embargo, debemos hacerle alguna mejora para que se sienta realmente como una entrega de proyecto final, aquí explicaremos nuestro modelo matemático que seguiremos para llegar a nuestro entregable final:

1. Problemática:

Nuestra pluma funciona perfectamente pero decidimos preguntarle al dueño de la mansión si hay algún cambio que le gustaría que tuviera su sistema para entrada y salida de su piso de exhibición, nos comentó que estaba encantado con el funcionamiento de su nuevo sistema pero que si le sería más sencillo si tuviera también una función para poder levantar la pluma usando alguna tarjeta que solamente tuviera él, y de esta manera sería mucho más seguro, así que nos pusimos manos a la obra.

2. Selección del modelo:

Incluir un sistema de tarjetas NFC va a requerir que nuestro proyecto sea más grande pero nuestro motor no tiene toda la fuerza del mundo, ¿entonces qué medida debería tener nuestra pluma para que el motor la soporte?, vamos a tener que resolverlo, para lo cual usamos la fórmula del torque máximo:

$$T_{\max} = \text{Fuerza} * \text{Gravedad} * \text{Movimiento}$$

$$T_{\max} = 1.8 \text{ kg/cm} * 9.8 \text{ m/s}^2 * 0.01 \text{ m/cm}$$

$$T_{\max} = 0.1764 \text{ Nm}$$

Este es el torque máximo que nuestro motor puede soportar, entonces más adelante en nuestro modelo vamos a usar la fórmula para el torque requerido, en lugar del torque máximo y de esa manera sabremos el tamaño adecuado para nuestra nueva pluma:

$$T_{\text{Requerido}} = (L / 2) * m * g * \cos(\theta)$$

Donde:

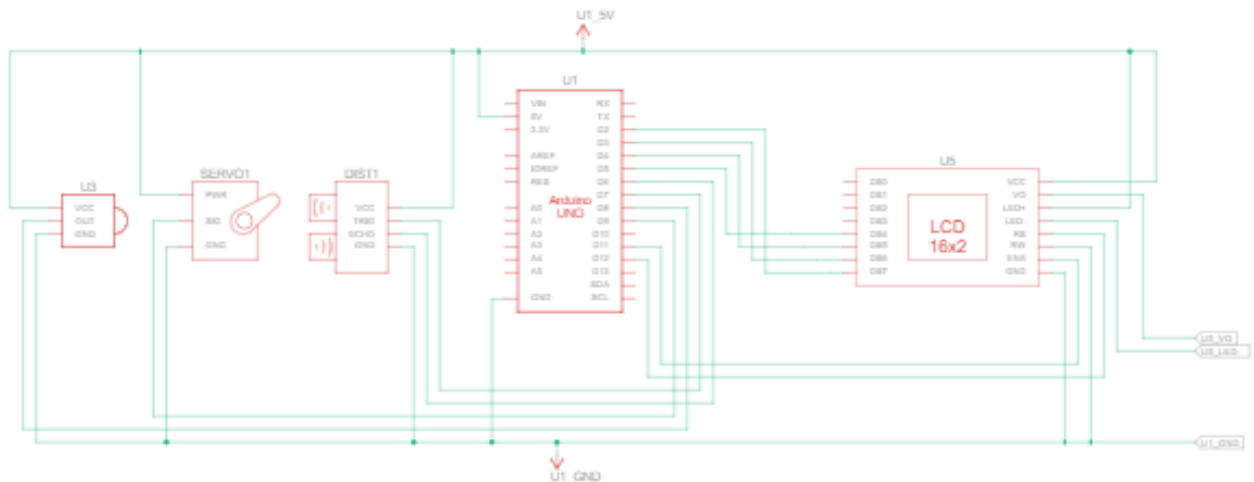
L -> Longitud de pluma (metros)

m -> Masa de la pluma (Kg)

$$g = 9.8 \text{ m/s}^2$$

$$\theta = 90^\circ \text{ (Ángulo)}$$

De momento hicimos el diagrama esquemático de nuestra nueva versión del proyecto:



3. Identificación de componentes.

Si hacemos la comparación con la primera versión realmente estamos usando los mismos componentes salvo por tres nuevos añadidos:

- Arduino uno R3: Es una placa electrónica de código abierto, tiene 14 pines digitales y 6 pines analógicos, trabaja con 5V, microcontrolador ATmega328P, velocidad de 16 MHz y RAM de 2 KB.
- Servomotor(SG90): Tiene un peso de 9 gramos, con una fuerza de 1.8 kg/cm, velocidad 0.1 s/60 grados, alimentación de 4.8 V (~5V) y funciona a una temperatura 0 °C – 55 °C.
- Sensor ultrasónico HC-SR04: Sirve para medir distancia por medio de pulsos de alta frecuencia que rebotan en los objetos, funciona con un voltaje de 5V, tiene un rango de medición de 2cm a 400 cm, frecuencia de 40KHz; cuenta con cuatro pines Vcc,, Trigger (Input), Echo (Output) y GND.
- Lector de tarjetas NFC: Módulo compatible con protocolos NFC (como el RC522) que opera a 3.3V o 5V. Tiene un alcance de lectura de 2–5 cm y autentifica tarjetas NFC registradas para activar el sistema.
- Tarjetas NFC: Tarjetas pasivas basadas en estándares como ISO 14443 (ej. NTAG213). Funcionan a 13.56 MHz y almacenan datos de identificación para autorizar el acceso al estacionamiento.
- Display LCD: Pantalla de pocos píxeles que nos servirá para mostrar mensajes al usuario final y que este entienda de una forma más didáctica el objetivo por el que está pasando lo que sea que esté pasando con el proyecto.

4. Cuantificación.

Ahora ya podemos proceder con la implementación de la fórmula que habíamos mencionado en la selección del modelo:

$$T_{\text{Requerido}} = (L / 2) * m * g * \cos(\theta)$$

$$(L / 2) * m * g \leq T_{\max}$$

$$L * m \leq [(2 * T_{\max}) / g]$$

$$L * m \leq [(2 * 0.1764) / 9.8]$$

$$L * m \leq 0.3528 / 9.8$$

$$L * m \leq 0.036$$

A partir de aquí ya podemos jugar con los valores de nuestra longitud y masa de la pluma y debemos cuidar que se mantengan 0.036, vamos a usar 3 ejemplos de tamaños diferentes que podremos tomar en cuenta a la hora de hacerlo en físico:

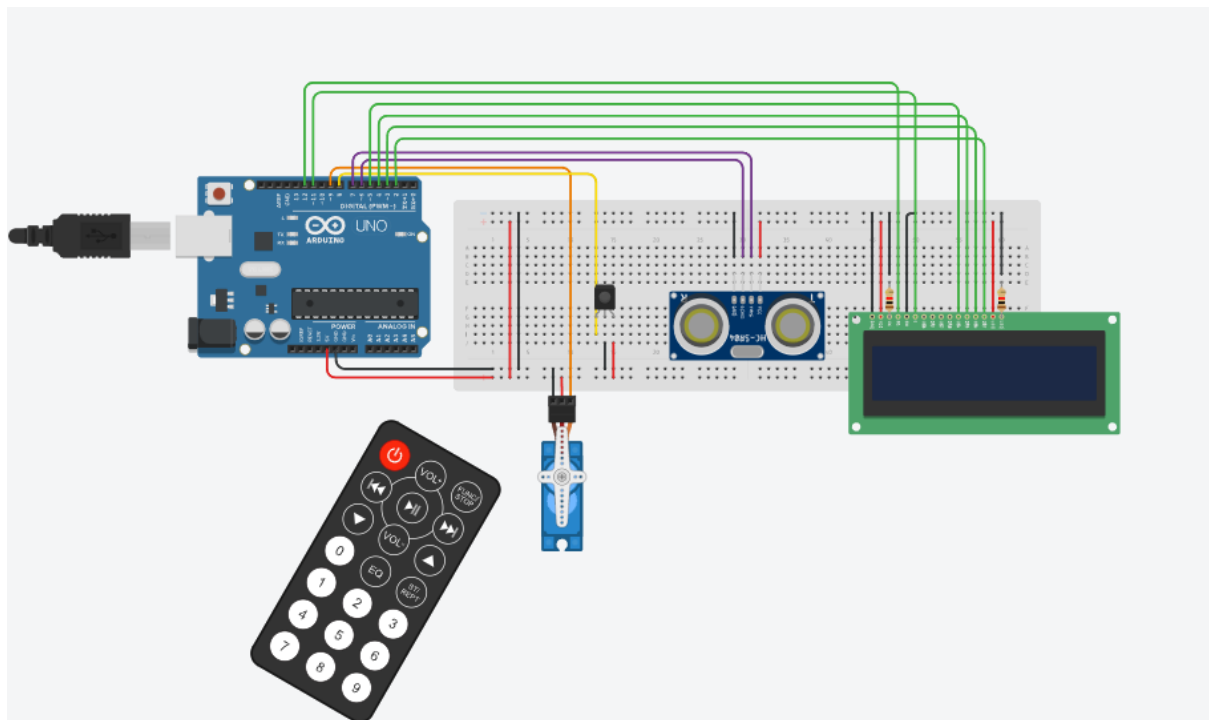
Longitud de la pluma	Masa máxima
0.3 m (30 cm)	$0.036/0.3 = 0.12 \text{ kg}$
0.2 m (20 cm)	$0.036/0.2 = 0.18 \text{ kg}$
0.15 m (15 cm)	$0.036/0.15 = 0.24 \text{ kg}$

De esta manera nos daremos una idea de cómo establecer las dimensiones de nuestra pluma y estaremos seguros de que no afectará al funcionamiento de nuestro motor.

5. Calibración.

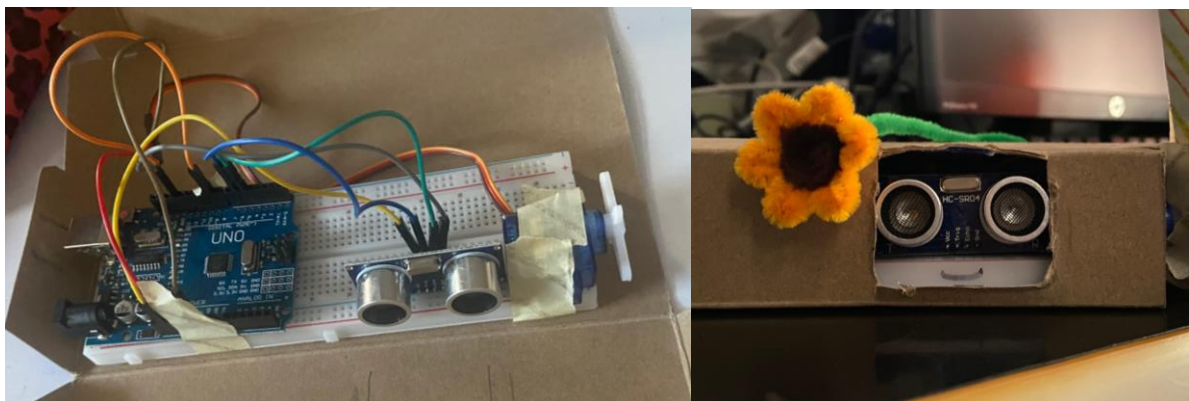
Para este punto ya estábamos trabajando con nuestra simulación, dentro de la misma acomodamos los componentes y los unimos a las entradas específicas del Arduino; no podemos usar un lector de tarjetas NFC en nuestra simulación, así que en su lugar usamos un control para activar y desactivar nuestro servomotor controlando la posición de la pluma, también incluimos una pantalla para que ahí se muestre un mensaje según la situación, probablemente mantengamos un mensaje de bienvenida cada vez que se levante la pluma; esta simulación nos permitió tener una idea más certera de lo que hará nuestro proyecto una vez que lo tengamos en físico.

Adjuntamos imagen del resultado de nuestra simulación:



6. Validación.

En cuanto al prototipado de nuestro proyecto, estamos tomando como referencia la versión anterior que entregamos para la actividad 4, de la cual adjuntamos imágenes en caso de que haga falta recordarle a la persona que corresponda el cómo se veía nuestro primer entregable:



Sin embargo, aún no tenemos en físico una nueva versión para el prototipo de nuestra pluma de estacionamiento mejorada, pero eso no quiere decir que ya está todo terminado para nosotros, con el transcurso de los días estaremos creando un prototipo v2 que complemente y a la vez aplique el contenido especificado en este modelo matemático.

7. Análisis de sensibilidad.

Al contar únicamente con la versión 1 del prototipo en nuestras manos no han habido cambios en nuestro código, el cuál adjuntamos a continuación, sin embargo, ya tenemos definidos los cambios que harán falta para que nuestra nueva versión de la pluma ni tope a la versión anterior, primeramente hará falta crecer el proyecto, de forma literal, hacerlo más grande, para lo cuál usaremos los valores que adquirimos en nuestro cuarto paso del presente modelo matemático, seguido de eso también implementaremos nuestro sistema de lectura de tarjetas NFC para la activación de la pluma de una forma más manual, del cual ya hablamos en nuestro quinto paso, y finalmente, añadiremos un Display LCD justo como se veía en nuestra simulación, esto para hacer más estético nuestro proyecto final a la vez que le añadimos funcionalidad.

- Código:

```
#include <Servo.h>

const int EchoPin = 5;
const int TriggerPin = 6;
const int ServoPin = 9; // Pin de control del servomotor
const int UmbralDistancia = 20; // Distancia en cm para activar el servo

Servo miServo; // Objeto para controlar el servo

void setup() {
  Serial.begin(9600);
  pinMode(TriggerPin, OUTPUT);
  pinMode(EchoPin, INPUT);
  miServo.attach(ServoPin); // Asignamos el pin al servo
  miServo.write(180); // Posición inicial en 0°
}

void loop() {
  int cm = ping(TriggerPin, EchoPin);
  Serial.print("Distancia: ");
  Serial.println(cm);

  if (cm > 0 && cm <= UmbralDistancia) { // Si detecta un objeto a menos de
    'UmbralDistancia' cm
    miServo.write(90); // Mueve el servo a 90°
    delay(2000);
  } else {
    miServo.write(180); // Regresa a 0°
  }

  delay(1000);
}

int ping(int TriggerPin, int EchoPin) {
```

```
    long duration, distanceCm;

    digitalWrite(TriigerPin, LOW);
    delayMicroseconds(4);
    digitalWrite(TriigerPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(TriigerPin, LOW);

    duration = pulseIn(EchoPin, HIGH);
    distanceCm = duration * 10 / 292 / 2;

    return distanceCm;
}
```

□